

xG Model Comparison (WIP)

Rento Saijo

Department of Mathematics, Connecticut College

February 7, 2026

Disclosure

See [compare.R](#) for the raw script to reproduce the results.

I. Introduction

Expected goals (xG) models estimate the probability that an individual shot becomes a goal, conditional on observable information at the moment of the shot. In hockey analytics, xG serves two related purposes: (i) evaluating shot quality (and shot generation/shot suppression) beyond raw shot counts, and (ii) providing a probabilistic baseline for comparing finishing and goaltending outcomes against expectation. This report compares two families of xG models:

1. Generalized Linear Models (Logistic Regression)

These models estimate

$$\Pr(\text{Goal} \mid \mathbf{x}) = \text{logit}^{-1}(\beta_0 + \beta^\top \mathbf{x}),$$

where $\mathbf{x}$ is a vector of shot- and game-level covariates. Logistic regression provides an interpretable baseline: each predictor contributes additively on the log-odds scale, and model complexity is controlled by the choice of features rather than by flexible nonlinear structure.

2. Gradient-boosted Decision Trees (LightGBM).

These models estimate the same probability, but using an ensemble of boosted decision trees. Compared to logistic regression, LightGBM can capture nonlinear effects, sharp thresholds, and higher-order interactions without manual feature engineering. In practice, this often improves discrimination and log-loss when relationships are not well-approximated by a linear log-odds function.

A. Situation Partitioning

Shots are first divided into four **mutually exclusive** and **collectively exhaustive** buckets, so that each shot is modeled using a specialized estimator appropriate to its context: standard 5-on-5 shots, special teams (non-empty net) shots, shootout/penalty shots, and empty net shots. The motivation is that shot-generation mechanisms (and defender/goalie configurations) differ substantially across various contexts; training a single global model tends to blur these regimes and can degrade probability estimates.

B. Feature Sets and Versioning

Within each situation bucket, models are trained in four “versions,” intended to incrementally add information beyond purely shot-location and shot-type descriptors. The versions are defined by which predictors are permitted:

- **Version 1: “Easily Controllable” (Shot Characteristics)**

Uses only variables that primarily reflect the shooter’s choice and the shot’s geometry:

- `distance`, `angle`, `shotType`

- **Model 2: Pre-shot Movement**

Adds variables describing immediate pre-shot dynamics:

- `dDdT`, `dAdT`, `isRush`, `isRebound`
- Note that `dDdT` and `dAdT` denotes the change in distance and angle over change in time from the previous event, respectively.

- **Model 3: Game Context**

Adds coarse situational context and scoreboard state (home/away, time, and score):

- `isHome`, `isPlayoff`, `period`, `secondsElapsedInPeriod`
- `goalsFor`, `goalsAgainst`, `SOGFor`, `SOGAgainst`, `fenwickFor`, `fenwickAgainst`, `corsiFor`, `corsiAgainst`

- **Model 4: Biometrics**

Adds shooter and goalie attributes:

- shooter: `shooterHeight`, `shooterWeight`, `shooterAge`, `shooterSide`, `shooterPositionCode`
- goalie: `goalieHeight`, `goalieWeight`, `goalieAge`, `goalieSide`

Because the four situation buckets do not share identical information content, some predictors are intentionally omitted where they are not meaningful or not available. For example, pre-shot movement features are not defined for shootouts/penalty shots in the same way as for in-play events, and empty-net shots do not involve an in-net goalie, so goalie biometrics are not applicable there. Also, empty-net status and skater counts (“for” and “against”) are included in applicable models (e.g., special teams models).

C. Training Data and Evaluation

All models are trained on NHL play-by-play shot events (`goal`, `shot-on-goal`, `missed-shot`) from multiple seasons (in particular, 2022-2023 to 2024-2025 seasons). In the comparisons that follow, we evaluate model outputs using likelihood- and calibration-sensitive metrics:

- **Log-loss (Cross-entropy):** penalizes overconfident incorrect probabilities and rewards well-calibrated sharp predictions. In practice, lower log-loss suggests the model is assigning *better probabilities*, especially on “borderline” chances where being too confident in either direction is costly.
- **Brier Score:** mean squared error for probabilistic forecasts. This is a more “average error” view: lower Brier scores indicate probabilities are closer to the true outcomes, and it can help distinguish whether improvements come from broadly reducing error rather than only avoiding catastrophic overconfidence.
- **Calibration (Observed / Expected):** compares total goals to total predicted xG. Values near 1 indicate good aggregate calibration; values > 1 suggest the model is underpredicting goals overall, and values < 1 suggest overprediction. This is especially informative because xG is often interpreted through season-level totals.

We compare these metrics across three evaluation seasons with different relationships to the training window:

- **2025–2026:** unseen future data (out-of-sample forward test)
- **2023–2024:** seen data (within the training window)
- **2021–2022:** unseen past data (out-of-sample backward test)

Metrics are reported both **across all situations** and restricted to **5-on-5 only**, since 5-on-5 constitutes the most common state and is often the primary target for player and team evaluation. Finally, note that the logistic regression “Version 4” model is omitted in the current draft due to numerical instability from quasi-separation in some contexts; the remaining logistic versions and all LightGBM versions are included for the comparisons below.

II. Calculation

Let us set up tikZ, load libraries, and define constants and helpers.

```
# Set up tikZ.
knitr::opts_chunk$set(
  echo      = TRUE,
  message   = FALSE,
  warning    = FALSE,
  dev       = 'tikz',
  sanitize  = FALSE,
  fig.align = 'center'
```

```

)

# ----- Setup ----- #

# Load libraries.
suppressMessages(library(glue))
suppressMessages(library(tidyverse))
suppressMessages(library(nhlscraper))

# Define constant.
SEASONS <- c(20212022, 20232024, 20252026)

# ----- Helpers ----- #

read_lgbm_xg <- function(season, version) {
  readr::read_csv(
    glue::glue('../data/shots_{season}_v{version}.csv'),
    show_col_types = FALSE
  ) %>%
  dplyr::transmute(
    gameId,
    eventId,
    !!rlang::sym(glue::glue('xG_v{version}_lightgbm')) := xG
  )
}

calc_metrics <- function(df, season_label) {
  xg_cols <- setdiff(
    names(df),
    c('gameId', 'eventId', 'situationCode', 'typeDescKey')
  )
  df_long <- df %>%

```

```

dplyr::mutate(y = dplyr::if_else(typeDescKey == 'goal', 1, 0)) %>%
tidyr::pivot_longer(
  cols = dplyr::all_of(xg_cols),
  names_to = 'model',
  values_to = 'p'
) %>%
dplyr::filter(!is.na(p)) %>%
dplyr::mutate(p = pmin(pmax(p, 1e-15), 1 - 1e-15))
summarise_scores <- function(d) {
  d %>%
    dplyr::summarise(
      season = season_label,
      goals = sum(y),
      n = dplyr::n(),
      total_xg = sum(p),
      calibration = goals / total_xg,
      brier = mean((p - y)^2),
      log_loss = -mean(y * log(p) + (1 - y) * log(1 - p)),
      .by = 'model'
    )
}
all_situations <- summarise_scores(df_long) %>%
  dplyr::mutate(scope = 'all')
five_on_five <- summarise_scores(df_long %>% dplyr::filter(situationCode == '1551')) %>%
  dplyr::mutate(scope = '5v5')
dplyr::bind_rows(all_situations, five_on_five) %>%
  dplyr::select(season, scope, model, n, goals, total_xg, calibration, brier, log_loss)
}

```

Let us load our shots data with populated xG from our 8 (7) models.

```

# Load data.
for (season in SEASONS) {
  v1_lightgbm <- read_lgbm_xg(season, 1)

```

```

v2_lightgbm <- read_lgbm_xg(season, 2)
v3_lightgbm <- read_lgbm_xg(season, 3)
v4_lightgbm <- read_lgbm_xg(season, 4)
shots <- nhlscraper::gc_pbps(season) %>%
  dplyr::filter(!(period == 5 & gameId == 5)) %>%
  nhlscraper::calculate_xG(model = 1) %>% dplyr::mutate(xG_v1_logistic = xG) %>%
  nhlscraper::calculate_xG(model = 2) %>% dplyr::mutate(xG_v2_logistic = xG) %>%
  nhlscraper::calculate_xG(model = 3) %>% dplyr::mutate(xG_v3_logistic = xG) %>%
  nhlscraper::calculate_xG(model = 4) %>% dplyr::mutate(xG_v4_logistic = xG)
shots_season <- shots %>%
  dplyr::left_join(v1_lightgbm, by = c('gameId', 'eventId')) %>%
  dplyr::left_join(v2_lightgbm, by = c('gameId', 'eventId')) %>%
  dplyr::left_join(v3_lightgbm, by = c('gameId', 'eventId')) %>%
  dplyr::left_join(v4_lightgbm, by = c('gameId', 'eventId')) %>%
  dplyr::filter(typeDescKey %in% c('goal', 'shot-on-goal', 'missed-shot')) %>%
  dplyr::select(
    gameId, eventId, situationCode, typeDescKey, xG_v1_logistic, xG_v2_logistic,
    xG_v3_logistic, xG_v4_logistic, xG_v1_lightgbm, xG_v2_lightgbm,
    xG_v3_lightgbm, xG_v4_lightgbm
  )
assign(glue::glue('shots_{season}'), shots_season, envir = globalenv())
}

# Drop logistic regression v4 (needs fixing).
shots_20212022 <- dplyr::select(shots_20212022, -xG_v4_logistic)
shots_20232024 <- dplyr::select(shots_20232024, -xG_v4_logistic)
shots_20252026 <- dplyr::select(shots_20252026, -xG_v4_logistic)

```

Let us calculate several model comparison metrics.

```

# Calculate model comparison metrics.
model_metrics <- dplyr::bind_rows(
  calc_metrics(shots_20212022, '20212022'),

```

```

calc_metrics(shots_20232024, '20232024'),
calc_metrics(shots_20252026, '20252026')
)

```

III. Analysis

Let us summarize model performance using log-loss, Brier score, and calibration (Observed / Expected xG), reported for both 5-on-5 only and all situations.

A. 5-on-5 Situation

2025–2026 (unseen future)

Model	n	goals	total_xg	cal	brier	log_loss
Logit V1	54,668	3,325	2,359.2	1.409	0.05549	0.21477
Logit V2	54,668	3,325	2,358.9	1.410	0.05541	0.21424
Logit V3	54,668	3,325	2,362.9	1.407	0.05539	0.21408
LightGBM V1	54,668	3,325	3,377.2	0.985	0.05399	0.20335
LightGBM V2	54,668	3,325	3,367.4	0.987	0.05362	0.20129
LightGBM V3	54,668	3,325	3,371.4	0.986	0.05362	0.20106
LightGBM V4	54,668	3,325	3,375.1	0.985	0.05362	0.20087

LightGBM is clearly better than logistic regression on both log-loss and Brier. The best log-loss is LightGBM V4, while LightGBM V2/V3/V4 are effectively tied on Brier at the reported precision. Logistic models have calibration around 1.41, indicating substantial underprediction of total xG relative to goals.

2023–2024 (seen)

Model	n	goals	total_xg	cal	brier	log_loss
Logit V1	95,428	5,643	3,941.5	1.432	0.05365	0.20748
Logit V2	95,428	5,643	3,933.1	1.435	0.05357	0.20697
Logit V3	95,428	5,643	3,931.6	1.435	0.05355	0.20687
LightGBM V1	95,428	5,643	5,535.1	1.019	0.05194	0.19467

Model	n	goals	total_xg	cal	brier	log_loss
LightGBM V2	95,428	5,643	5,495.8	1.027	0.05065	0.18740
LightGBM V3	95,428	5,643	5,498.5	1.026	0.05131	0.19211
LightGBM V4	95,428	5,643	5,494.4	1.027	0.05114	0.19108

On 5-on-5 in this season, LightGBM V2 is best on both log-loss and Brier. Relative to V2, V3/V4 are slightly worse on these metrics here, suggesting the largest lift comes from the pre-shot movement features, while the extra context/biometrics do not add further improvement in this slice. Logistic calibration remains far from 1.

2021–2022 (unseen past)

Model	n	goals	total_xg	cal	brier	log_loss
Logit V1	95,540	5,842	4,145.8	1.409	0.05546	0.21431
Logit V2	95,540	5,842	4,165.9	1.402	0.05553	0.21481
Logit V3	95,540	5,842	4,166.2	1.402	0.05552	0.21474
LightGBM V1	95,540	5,842	5,878.2	0.994	0.05453	0.20704
LightGBM V2	95,540	5,842	5,904.8	0.989	0.05479	0.20921
LightGBM V3	95,540	5,842	5,892.3	0.991	0.05466	0.20793
LightGBM V4	95,540	5,842	5,918.3	0.987	0.05465	0.20776

LightGBM remains best overall, but LightGBM V1 is best on this older out-of-sample season (both log-loss and Brier). This is consistent with a robustness vs. richness tradeoff: additional features can fit era-specific patterns that do not transfer as well across larger time gaps.

B. All Situations

2025–2026 (unseen future)

Model	n	goals	total_xg	cal	brier	log_loss
Logit V1	70,971	5,210	3,801.9	1.370	0.06375	0.23951
Logit V2	70,971	5,210	3,807.2	1.368	0.06367	0.23902
Logit V3	70,971	5,210	3,806.0	1.369	0.06366	0.23894

Model	n	goals	total_xg	cal	brier	log_loss
LightGBM V1	70,971	5,210	5,442.0	0.957	0.06226	0.22842
LightGBM V2	70,971	5,210	5,431.5	0.959	0.06183	0.22631
LightGBM V3	70,971	5,210	5,442.3	0.957	0.06182	0.22609
LightGBM V4	70,971	5,210	5,430.5	0.959	0.06187	0.22617

Across all situations, LightGBM V3 has the best log-loss with LightGBM V2 extremely close (and essentially tied on Brier at the reported precision). Calibration for LightGBM is slightly below 1 (around 0.96), indicating mild aggregate overprediction for this season, while logistic regression remains strongly underpredictive.

2023–2024 (seen)

Model	n	goals	total_xg	cal	brier	log_loss
Logit V1	123,136	8,772	6,190.8	1.417	0.06196	0.23240
Logit V2	123,136	8,772	6,187.3	1.418	0.06187	0.23190
Logit V3	123,136	8,772	6,186.9	1.418	0.06185	0.23179
LightGBM V1	123,136	8,772	8,700.2	1.008	0.05961	0.21815
LightGBM V2	123,136	8,772	8,659.8	1.013	0.05848	0.21184
LightGBM V3	123,136	8,772	8,657.2	1.013	0.05888	0.21503
LightGBM V4	123,136	8,772	8,649.8	1.014	0.05808	0.21265

In this season, LightGBM V2 has the best log-loss, while LightGBM V4 has the best Brier score. Both are substantially ahead of the logistic baselines. Calibration for LightGBM is near 1 (slight underprediction at about 1.01), while logistic regression remains heavily underpredictive.

2021–2022 (unseen past)

Model	n	goals	total_xg	cal	brier	log_loss
Logit V1	122,350	8,936	6,335.6	1.410	0.06316	0.23743
Logit V2	122,350	8,936	6,360.4	1.405	0.06324	0.23789
Logit V3	122,350	8,936	6,365.6	1.404	0.06323	0.23783
LightGBM V1	122,350	8,936	8,931.1	1.001	0.06198	0.22898

Model	n	goals	total_xg	cal	brier	log_loss
LightGBM V2	122,350	8,936	8,960.6	0.997	0.06224	0.23101
LightGBM V3	122,350	8,936	8,949.4	0.999	0.06211	0.22992
LightGBM V4	122,350	8,936	8,974.3	0.996	0.06219	0.22999

As with 5v5, LightGBM V1 is best on the older out-of-sample season across all situations. Logistic regression remains substantially under-calibrated, while LightGBM stays extremely close to 1.

C. Summary

1. Model family matters more than version. Across every season and scope, LightGBM achieves substantially better log-loss and Brier score than logistic regression.
2. Within LightGBM, the best version depends on season and scope. For 5-on-5, the best log-loss is V4 (2025–2026), V2 (2023–2024), and V1 (2021–2022). For all situations, the best log-loss is V3 (2025–2026), V2 (2023–2024), and V1 (2021–2022). Differences among V2/V3/V4 are often small compared to the gap between LightGBM and logistic regression.
3. Calibration differences are practically important, and logistic outputs can likely be recalibrated. Logistic models consistently have Observed/Expected well above 1, meaning they underpredict xG totals relative to goals. At the same time, their log-loss is not catastrophically worse than LightGBM, which suggests the logistic models are ranking chances in a broadly reasonable way but are mis-scaled (their probabilities are shifted downward overall, or compressed away from the true base rate). Potential fixes include:
 - Platt-style recalibration: fit a 1D logistic regression of the form $\text{logit}(y) \sim a + b \text{logit}(\hat{p})$, then use $\hat{p}_{\text{cal}} = \text{logit}^{-1}(a + b \text{logit}(\hat{p}))$.
 - Isotonic regression: a nonparametric monotone map $\hat{p} \mapsto \hat{p}_{\text{cal}}$ that can correct more flexible miscalibration patterns.
 - Simple intercept adjustment: if the issue is mostly an overall base-rate shift, updating only the intercept on the evaluation era can move total_xg toward goals.

This would bring logistic season-level totals closer to reality while preserving interpretability, and it makes the logistic models more viable as a baseline when probabilistic calibration is the primary concern. I'd like to work on this in the near future.

4. 5-on-5 and all-situations tell a consistent story, but reveal tradeoffs. The overall ordering is stable (LightGBM far ahead of logistic), and the all-situations results highlight small metric-dependent differences among LightGBM variants in the seen season.

IV. Conclusion

This comparison suggests a simple takeaway: LightGBM is the best current option for predictive performance, while logistic regression remains a useful, interpretable baseline that likely needs post-hoc calibration to be reliable for season-level xG totals. Across all seasons and both scopes (5-on-5 and all situations), LightGBM consistently achieves lower log-loss and Brier scores. The best LightGBM version varies by slice: V2–V4 tend to win on modern seasons, while the simpler V1 looks more robust when transferred further backward in time. The clearest gap between model families is calibration. Logistic models underpredict goals in aggregate (Observed/Expected roughly 1.37–1.44), so raw totals are misleading without adjustment. Yet logistic log-loss is not dramatically worse than LightGBM, which is consistent with “mostly right ordering, wrong scale”: chances are ranked reasonably, but probabilities are shifted downward and/or too compressed. That kind of mis-scaling is well-suited to recalibration on a held-out set via (i) Platt-style correction on the logit scale, (ii) isotonic regression, or (iii) an intercept-only update if the main issue is base-rate drift; recalibration would preserve interpretability while bringing totals closer to realized goal rates. For this WIP stage, a reasonable production-facing recommendation is:

- Use LightGBM V2 as a strong default for modern-season accuracy, and consider V1 as a robustness-oriented baseline when transferring across larger time gaps.
- Keep logistic regression as an interpretable reference model, but treat its raw probabilities as uncalibrated; apply post-hoc calibration if it is to be used for total-xG interpretation or downstream decision-making.

Next steps are to (i) add formal calibration curves (reliability diagrams) and slope/intercept diagnostics by season and situation bucket, (ii) implement and evaluate the recalibration layer for logistic regression using a strict train/calibration/test split, and (iii) check whether the remaining miscalibration is mostly global (single mapping) or situation-specific (requiring separate recalibrators per bucket). Thank you for taking the time to read this.